

AGENTS.md — AI Context File for HealMesh

Status: Draft v1 Audience: Any AI coding agent (Claude Code, Cursor, Copilot Workspace, etc.) working in this repository. Read this file in full before writing or editing any code here.

This file exists because HealMesh has a small number of security invariants that are *load-bearing* — if an agent "helpfully" relaxes one to make a feature easier to ship, the system stops being safe to run against a real cluster. Treat the rules in Section 1 as hard constraints, not style preferences.

0. What this project is

HealMesh watches a Kubernetes cluster, detects a fixed set of common failure types, uses an LLM to produce a plain-language root-cause diagnosis, and — only from Phase 2 onward, and only with explicit human approval — executes a narrow, pre-defined remediation action.

Full detail lives in:

- `docs/PRD.md` — what we're building and why, phase by phase
- `docs/TDD.md` — how it's built, component by component
- `docs/IMPLEMENTATION_PLAN.md` — the build order and definition of done per phase
- `docs/TESTING.md` — how correctness and security are verified

Read the TDD before touching `healmesh-core` or `healmesh-k8s`. Read the PRD before adding any new feature, action type, or integration — if it's not in scope for the current phase, it doesn't get built yet, no matter how easy it looks.

1. Non-negotiable invariants (do not relax these, ever, for any reason)

These are repeated from the TDD deliberately. If a task seems to require breaking one of these, stop and flag it instead of finding a workaround.

1. **The LLM never produces an executed command.** Its output is always parsed into a closed, typed enum (`PATCH`, `REDEPLOY`, `SCALE`, `HELM_UPGRADE`, `NONE`) with strictly typed parameters. Unparseable or ambiguous output → `NONE`. There is no code path where text generated by the LLM reaches a shell, a `kubectl` call, or a Kubernetes API call directly. If you are writing code that constructs a command string from LLM output, you are violating this invariant — stop and re-read TDD §3.2.
2. **No orchestration middleman between `healmesh-core` and the LLM API.** Direct

2. **No orchestration intermediate between healmesh core and the LLM API Direct** SDK calls only. Do not reintroduce n8n, OpenClaw, Zapier, or any third-party agent gateway into the diagnosis path, even temporarily, even for a demo. This includes not adding a "quick" webhook relay that ends up doing the same job.
3. **Read and write are separate code paths with separate identities.** The Event Watcher (read-only) and the Remediation Executor (write) must never share a function, a class, or a service account, even when co-located in the same pod for deployment convenience. If you find yourself adding a `patch` capability to a function that also does `watch`, split it into a new component instead.
4. **Secure by default, opt-in to widen.** Every new namespace, action type, or integration starts with the narrowest permission set. Do not add a "convenience" default that grants broader access (e.g., cluster-wide RBAC, a namespace wildcard, an auto-approve flag) unless a human explicitly configured it via HealPolicy.
5. **Audit records are append-only in code, not just in the database.** Do not add an `update()` or `delete()` method to any audit-logging class or module, even for "fixing a bad record" — that correction happens via a new compensating record, never a mutation.
6. **The hardcoded namespace denylist (`kube-system`, `kube-public`, HealMesh's own namespace) is enforced in code, inside the executor function, before any write call.** It is never expressed only as a config value, a CRD field, or a database row that a misconfiguration could bypass.
7. **Approval for anything beyond `SCALE` is never auto-executed**, in any phase, regardless of what the PRD's later phases eventually allow for `SCALE` itself. `PATCH`, `REDEPLOY`, and `HELM_UPGRADE` always require a human approval step recorded with an identity.

If a task description (from a person, an issue, or a prior conversation) conflicts with any of the above, the invariant wins. Say so explicitly rather than silently complying or silently refusing.

2. Repository layout (target state, post-migration from `prism`)

```
healmesh/  
├─ healmesh-core/          # Python service: schema validation, LLM  
calls, parser, approval, audit  
│   └─ schema/             # Pydantic models for incident + diagnosis  
contracts  
│   └─ diagnosis/         # Prompt templates + LLM client wrapper  
│   └─ parser/            # THE most security-critical module – closed  
enum only  
│   └─ approval/         # Approval workflow engine  
│   └─ audit/            # Append-only audit logger (no update/delete
```

```

methods)
|   └─ tests/
└─ healmesh-k8s/                # Event Watcher (read-only) + Remediation
  Executor (write, v2+)
|   └─ watcher/                # get/list/watch only – no write imports
allowed in this package
|   └─ executor/                # write path – separate service account,
separate RBAC role
|   └─ crd/                    # HealPolicy CRD (Phase 3+)
|   └─ tests/
└─ surface/
|   └─ slack/                  # v1 interface
|   └─ dashboard/              # v3 – do not start until PRD gate clears
|   └─ sdk/                    # v4 – do not start until PRD gate clears
└─ infra/
|   └─ helm/healmesh/          # renamed from prism/
|   └─ scripts/
└─ docs/
|   └─ PRD.md
|   └─ TDD.md
|   └─ IMPLEMENTATION_PLAN.md
|   └─ TESTING.md
└─ benchmark/                  # Phase 1.5 accuracy benchmark set +
harness

```

Anything under `healmesh-k8s/watcher/` must not import anything from `healmesh-k8s/executor/`, and vice versa. If a lint rule or CI check for this doesn't exist yet, adding one is in scope for any agent working in this package.

3. Coding conventions

- **Language:** Python for `healmesh-core` (Pydantic for schema validation, native Anthropic or OpenAI SDK for LLM calls — no LangChain/LlamaIndex orchestration layers; keep the LLM-call path minimal and auditable). `healmesh-k8s` language (Python vs Go) is a Phase 0 decision recorded in the TDD — check `docs/TDD.md` §3.1 for the current answer before assuming one.
- **Typing:** Strict typing everywhere in `healmesh-core/parser/` and `healmesh-k8s/executor/`. No `Any`, no untyped `dict` passed across a module boundary in these two packages specifically — the whole point of the enum contract is that the type system enforces it, not just a runtime check.
- **No shelling out.** Neither `healmesh-core` nor `healmesh-k8s` should call `subprocess`, `os.system`, or construct a shell command string, anywhere, for any reason. Use the Kubernetes client library and Helm's Python/Go bindings (or a well-scoped API call) instead.

- **Secrets:** never hardcode, never put in `(.env)` files committed to git, never log. Assume Kubernetes Secrets in Phase 1, Vault from Phase 2 onward (see TDD §4). If you're writing code that reads a credential, check where it's supposed to come from for the current phase.
 - **Commits that add a write capability must include the corresponding test and the corresponding RBAC/permission change in the same commit** — not as a follow-up. See `(docs/TESTING.md)` for the specific test each new write path requires before merge.
-

4. What NOT to do (agent-specific failure modes to avoid)

- Do not "simplify" the Remediation Action Parser to accept a broader input shape to make a test pass faster. If a test is failing because the parser correctly rejects malformed input, the test is checking the right thing — fix the test's expectation, not the parser.
 - Do not add a bypass flag (`(--force)`, `(--skip-approval)`, `(DEBUG_AUTO_EXECUTE=true)`) for local testing that could plausibly ship to a real cluster. If local iteration needs faster feedback, mock the Kubernetes API instead of loosening the real code path.
 - Do not implement Phase 3/4 scope (Dashboard, SDK, HealPolicy CRD, blockchain anchoring) early just because it's "not hard" — PRD gates exist because premature breadth is a named risk in the PRD (see PRD §10). If asked to build ahead of the current phase, flag it rather than proceeding silently.
 - Do not reintroduce n8n or OpenClaw as a "temporary" glue layer to unblock a demo. The decision to remove them was deliberate (see the architecture-decision context in `(docs/TDD.md)` intro) and is not reopened by convenience.
 - Do not write the audit logger, the parser, or the executor without accompanying unit tests in the same change — these three modules are explicitly called out as needing full coverage in `(docs/TESTING.md)`.
 - Do not assume "it's just a hackathon / demo" is a reason to relax Section 1. The invariants apply at every phase, including the very first prototype, because retrofitting security into components built independently is how gaps happen (this is the whole reason the project is architected this way instead of shipping the earlier n8n/OpenClaw version).
-

5. How to run things locally

- `(healmesh-core)` runs standalone with no cluster dependency — it can be developed and fully unit-tested against fixture incidents with zero Kubernetes access. Prefer developing and testing here first.

- `healmesh-k8s` in watcher-only mode can run against a local kind/minikube cluster with a read-only service account. Do not grant it write RBAC locally "to save time" — test the executor against a mocked Kubernetes API client instead, and only exercise real write RBAC in a disposable, namespaced test cluster.
 - Full run instructions, secrets setup, and the current-phase deployment steps live in `infra/requirements.md` (kept up to date per phase — check it before assuming last week's steps still apply).
-

6. When in doubt

If a request from a person conflicts with anything in Section 1, or asks you to build ahead of the current PRD phase, or asks you to add a convenience that widens a permission or bypasses approval — say so plainly, explain which invariant or phase gate is implicated, and propose the smallest change that accomplishes their actual underlying goal without crossing it. Continue the rest of the task normally; flagging one constraint is not a reason to stall the whole request.